

SATSET SUPER-APP SISTEM ADMINISTRASI TERPADU & SENTRALISASI ELEKTRONIK

Dosen Pengampu: Dr. Ir. Uung Ungkawa, M.T.

Bidang : Peningkatan Pelayanan Masyarakat

Asisten Lab :
- Delisya
- Riza Maulana
- Fathur

Disusun oleh :

15-2024-141 Robby Rizky Permana
15-2024-129 - Alsani Abdul Rofiq
15-2024-182 - Syafa Meisya Fitria
15-2024-174 - Kelvin Manuel

Kelompok – 04



Prodi Informatika
Fakultas Teknologi Industri
Institut Teknologi Nasional
Bandung
2025

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas berkat dan rahmat-Nya, Kelompok 4 dapat menyelesaikan proposal pengembangan proyek bertajuk **"SatSet SuperApp (Sistem Administrasi Terpadu & Sentralisasi Elektronik)**. Proposal ini disusun sebagai bagian dari pemenuhan proposal mata kuliah **IFB-202 Praktikum Pemrograman Berorientasi Objek** di program studi Informatika, **Institut Teknologi Nasional (Itenas) Bandung**.

Proyek ini bermula dari keresahan terhadap alur birokrasi pelayanan masyarakat yang sering kali dinilai lambat dan tidak efisien. Melalui pengembangan *backend* REST API & *frontend* UI/UX berbasis **Spring Boot**, kami berupaya menghadirkan solusi digital yang mampu mendigitalisasi serta mempercepat proses pengaduan warga dan layanan administrasi secara *real-time*.

Dalam penyusunan proposal ini, kami menyadari bahwa keberhasilan proyek ini tidak lepas dari kerja sama tim yang solid serta bimbingan dari berbagai pihak. Oleh karena itu, kami mengucapkan terima kasih kepada

1. Dosen pengampu mata kuliah Pemrograman Berorientasi Objek yang telah memberikan arahan teknis.
2. Seluruh Asisten Laboratorium mata kuliah Praktikum Pemrograman Berorientasi Objek.
3. Rekan-rekan Kelompok 4 yang telah berkontribusi dalam perancangan arsitektur sistem dan desain database.

Kami menyadari bahwa proposal ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan demi penyempurnaan proyek "SatSet" ini di masa mendatang agar dapat memberikan manfaat nyata bagi Masyarakat

DAFTAR ISI

BAB 1 PENDAHULUAN.....	4
1.1 Latar Belakang Project.....	4
1.2 Rumusan Masalah.....	4
1.3 Tujuan Sistem.....	4
1.4 Batasan Sistem.....	5
BAB II LANDASAN TEORI.....	6
2.1 Konsep Pemrograman Berorientasi Objek (OOP).....	6
2.1.1 Encapsulation.....	6
2.1.4 Abstraction.....	7
2.2 Teknologi yang Digunakan.....	7
2.2.1 Java 17+ & Spring Boot Framework.....	7
2.2.2 Spring Data JPA & PostgreSQL.....	7
2.2.3 REST API.....	8
2.3 Infrastruktur & Deployment.....	8
2.3.1 Mini PC Proxmox.....	8
2.3.2 Cloudflare Tunnel.....	8
BAB III PERANCANGAN SISTEM.....	9
3.1 Desain Database.....	9
3.1.1 Entity Relationship Diagram (ERD).....	9
3.1.2 Daftar Tabel Lengkap dan Atribut.....	10
3.1.3 Penjelasan Relasi Antar Tabel (One-to-Many & Many-to-Many).....	10
3.2 Desain OOP.....	11
3.2.1 Class Diagram (Minimal 5 Class).....	11
3.2.2 Penjelasan Hubungan Antar Class (Inheritance & Association).....	12
3.2.3 Identifikasi Komponen Spring (@Entity, @Service, @Controller, @Repository)..	12
3.3 Rancangan REST API.....	12
BAB IV PENUTUP.....	14
4.1 Kesimpulan.....	14
4.2 Saran.....	14

BAB 1

PENDAHULUAN

1.1 Latar Belakang Project

Dalam era digitalisasi saat ini, efisiensi pelayanan publik menjadi salah satu indikator keberhasilan tata kelola administrasi. Namun, pada kenyataannya, banyak proses birokrasi di masyarakat—seperti pelaporan fasilitas umum yang rusak, pengurusan surat-surat administrasi, hingga sistem antrian—masih bersifat konvensional atau terfragmentasi. Hal ini sering kali mengakibatkan proses yang lambat, berbelit-belit, dan kurangnya transparansi status pelayanan bagi warga.

Ketidakterpaduan sistem informasi membuat warga kesulitan untuk memantau sejauh mana laporan atau pengajuan mereka diproses. Oleh karena itu, diperlukan sebuah sistem aplikasi utuh (*Superapp*) yang mampu memusatkan berbagai layanan administrasi ke dalam satu platform digital yang cepat dan terintegrasi. Proyek "**SatSet SuperApp**" hadir sebagai solusi aplikasi terpadu yang terdiri dari *backend* yang kuat dan antarmuka pengguna (*frontend*) yang intuitif untuk mendigitalisasi alur pelayanan masyarakat secara *real-time*.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, permasalahan yang ingin diselesaikan melalui proyek ini adalah:

1. Bagaimana membangun ekosistem aplikasi yang mengintegrasikan layanan pengaduan dan administrasi dari sisi pengguna hingga sistem pusat?
2. Bagaimana mengimplementasikan prinsip Pemrograman Berorientasi Objek (OOP) menggunakan Java Spring Boot untuk menciptakan *backend* yang skalabel?
3. Bagaimana merancang antarmuka pengguna (*frontend*) yang memudahkan user dalam melakukan pengajuan dan melacak status pelayanan secara *real-time*?

1.3 Tujuan Sistem

Tujuan utama dari pengembangan proyek ini adalah:

1. Membangun **REST API** sebagai fondasi data menggunakan framework Spring Boot.
2. Mengembangkan antarmuka **Frontend** yang responsif sebagai media interaksi *User* dan *admin*.
3. Mendigitalisasi alur pelayanan masyarakat, mulai dari pengajuan hingga penyelesaian laporan (*ticketing*).
4. Mengotomatisasi sistem antrian dan sentralisasi data pengaduan warga agar birokrasi menjadi lebih efisien.

5. Mengembangkan *feed information* (Mading Digital), di berbagai scenario.

1.4 Batasan Sistem

Agar pengembangan sistem tetap fokus dan terukur, ditetapkan batasan-batasan sebagai berikut:

- Pengembangan berfokus sepenuhnya pada sisi *backend* (API).
- Fitur utama mencakup modul pengaduan, layanan administrasi, riwayat pelayanan, dan *dashboard* admin.
- Pengerjaan ini tidak mencakup pembuatan antarmuka pengguna atau *UI Frontend*.
- Implementasi teknis mencakup penggunaan database PostgreSQL dan penerapan prinsip OOP (Encapsulation, Inheritance, Polymorphism, dan Abstraction)

BAB II

LANDASAN TEORI

2.1 Konsep Pemrograman Berorientasi Objek (OOP)

Pemrograman Berorientasi Objek (*Object-Oriented Programming*) adalah sebuah paradigma pemrograman yang berpusat pada konsep "objek" yang merepresentasikan entitas di dunia nyata. Objek-objek ini memiliki status (*state*) yang disimpan dalam atribut dan perilaku (*behavior*) yang didefinisikan dalam metode. Penerapan OOP bertujuan untuk meningkatkan fleksibilitas, keterawatan, dan skalabilitas kode program melalui empat pilar utama.

2.1.1 Encapsulation

Encapsulation atau pengkapsulan merupakan prinsip dasar yang berfungsi sebagai pelindung integritas data di dalam sebuah objek. Secara teknis, prinsip ini bekerja dengan cara menyembunyikan detail implementasi internal dan hanya mengizinkan akses ke data melalui antarmuka atau metode publik yang telah ditentukan. Dalam pengembangan *backend* SATSET, kelompok kami menerapkan *encapsulation* secara ketat pada kelas-kelas entitas seperti TiketLayanan. Atribut sensitif seperti *statusTiket* dan *nomorTiket* dideklarasikan dengan akses *modifier* *private* untuk mencegah manipulasi data secara ilegal dari luar kelas. Sebagai gantinya, kami menyediakan metode *getter* dan *setter* yang berfungsi sebagai pintu gerbang terkontrol, di mana validasi data dapat dilakukan sebelum perubahan status benar-benar disimpan ke dalam sistem.

2.1.2 Inheritance

Encapsulation atau pengkapsulan merupakan prinsip dasar yang berfungsi sebagai pelindung integritas data di dalam sebuah objek. Secara teknis, prinsip ini bekerja dengan cara menyembunyikan detail implementasi internal dan hanya mengizinkan akses ke data melalui antarmuka atau metode publik yang telah ditentukan. Dalam pengembangan *backend* SATSET, kelompok kami menerapkan *encapsulation* secara ketat pada kelas-kelas entitas seperti TiketLayanan. Atribut sensitif seperti *statusTiket* dan *nomorTiket* dideklarasikan dengan akses *modifier* *private* untuk mencegah manipulasi data secara ilegal dari luar kelas. Sebagai gantinya, kami menyediakan metode *getter* dan *setter* yang berfungsi sebagai pintu gerbang terkontrol, di mana validasi data dapat dilakukan sebelum perubahan status benar-benar disimpan ke dalam sistem.

2.1.3 Polymorphism

Polymorphism merupakan kemampuan suatu objek untuk mengambil berbagai bentuk yang berbeda melalui mekanisme antarmuka atau pendefinisian ulang metode. Pada arsitektur sistem ini, polimorfisme diimplementasikan melalui penggunaan *interface* pada lapisan layanan

yang mendefinisikan kontrak fungsi secara umum. Hal ini memungkinkan sistem untuk memiliki berbagai variasi implementasi logika bisnis tanpa harus mengubah struktur pemanggilan pada lapisan pengontrol, sehingga memberikan fleksibilitas tinggi apabila di masa depan terdapat perubahan aturan birokrasi yang perlu diterapkan.

2.1.4 Abstraction

Abstraction atau abstraksi digunakan untuk menyembunyikan detail implementasi teknis yang rumit dan hanya menonjolkan fungsi-fungsi esensial yang diperlukan oleh pengguna sistem. Dengan menerapkan abstraksi, kompleksitas pengolahan data di tingkat basis data maupun logika bisnis yang berlapis-lapis dapat disederhanakan menjadi rangkaian instruksi yang mudah dipahami. Konsep ini memastikan bahwa pengembang dapat berfokus pada interaksi antar komponen sistem secara makro tanpa harus terbebani oleh rincian teknis yang sangat spesifik di setiap bagian kode.

2.2 Teknologi yang Digunakan

2.2.1 Java 17+ & Spring Boot Framework

Pemilihan Java 17 sebagai bahasa pemrograman utama didasarkan pada statusnya sebagai versi *Long Term Support* yang menawarkan stabilitas dan dukungan keamanan jangka panjang. Versi ini menghadirkan berbagai penyempurnaan performa pada mesin virtual Java serta fitur-fitur bahasa modern yang memungkinkan penulisan kode menjadi lebih ringkas dan efisien. Penggunaan Java 17 memastikan bahwa sistem administrasi yang dibangun memiliki fondasi yang kuat untuk menangani beban kerja layanan publik yang terus meningkat.

Interaksi dengan basis data dikelola menggunakan Spring Data JPA yang bertindak sebagai jembatan antara objek Java dengan tabel-tabel relasional di dalam database. PostgreSQL dipilih sebagai sistem manajemen basis data utama karena reputasinya dalam menjaga integritas data serta kemampuannya menangani kueri kompleks dengan sangat cepat. Kombinasi kedua teknologi ini memungkinkan pengelolaan data pengaduan dan administrasi warga dilakukan secara konsisten dan aman dari risiko kehilangan informasi.

2.2.2 Spring Data JPA & PostgreSQL

Spring Boot digunakan sebagai kerangka kerja utama karena kemampuannya dalam mempercepat proses pengembangan melalui fitur konfigurasi otomatis yang cerdas. *Framework* ini mengadopsi arsitektur berlapis yang memisahkan tanggung jawab antara penanganan permintaan pengguna, pengolahan logika bisnis, dan akses ke penyimpanan data. Dengan dukungan ekosistem yang luas, Spring Boot memudahkan pengembang dalam mengintegrasikan berbagai modul keamanan dan pemantauan sistem yang sangat krusial bagi sebuah *superapp*.

2.2.3 REST API

Sistem ini dibangun dengan arsitektur REST yang menggunakan protokol HTTP sebagai standar komunikasi antar perangkat lunak di internet. Dengan mengadopsi gaya arsitektur ini, server *backend* SatSet dapat melayani permintaan dari berbagai platform baik aplikasi *mobile* maupun web melalui pertukaran data dalam format JSON yang ringan. Pendekatan ini menjamin bahwa sistem dapat terintegrasi dengan mudah dengan layanan pemerintah lainnya atau aplikasi pihak ketiga di masa depan tanpa kendala kompatibilitas.

2.3 Infrastruktur & Deployment

tinggi, diperlukan strategi infrastruktur yang Agar sistem SatSet dapat diakses secara luas oleh masyarakat dengan tingkat ketersediaan yang efisien namun tetap mengutamakan keamanan

2.3.1 Mini PC Proxmox

Seluruh lingkungan server dijalankan di atas platform virtualisasi Proxmox VE yang terpasang pada perangkat keras Mini PC. Teknologi ini memungkinkan pembagian sumber daya perangkat keras menjadi beberapa lingkungan terisolasi yang dapat dikelola secara independen, sehingga meningkatkan reliabilitas sistem secara keseluruhan. Dengan Proxmox, proses pencadangan data dan pemulihan sistem jika terjadi kegagalan dapat dilakukan dengan jauh lebih cepat dan terstruktur.

2.3.2 Cloudflare Tunnel

Untuk menghubungkan server lokal ke jaringan internet global secara aman, digunakan layanan Cloudflare Tunnel yang menciptakan jalur enkripsi khusus tanpa perlu membuka jalur komunikasi publik yang berisiko. Metode ini memberikan lapisan perlindungan tambahan terhadap berbagai ancaman siber seperti serangan *DDoS* atau akses ilegal dari luar. Melalui teknologi ini, warga dapat mengakses layanan administrasi SatSet kapan saja dan di mana saja dengan jaminan keamanan data yang tetap terjaga.

BAB III

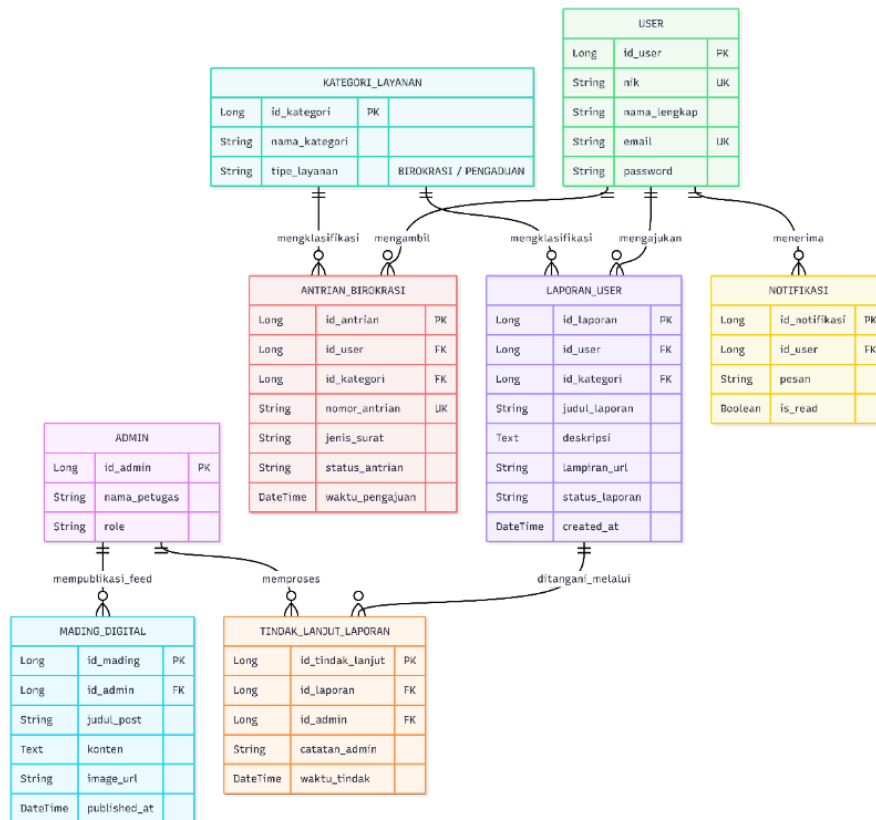
PERANCANGAN SISTEM

3.1 Desain Database

Perancangan basis data dalam ekosistem "SatSet Superapp" merupakan fondasi paling krusial untuk menjamin bahwa seluruh data administrasi warga dapat dikelola dengan tingkat integritas yang tinggi. Sistem ini menggunakan basis data relasional PostgreSQL yang dirancang untuk menangani beban data dinamis, mulai dari informasi profil penduduk hingga pelacakan kronologis pengaduan masyarakat. Fokus utama dari desain ini adalah menciptakan skema yang mampu meminimalisir redundansi data serta memastikan setiap entitas memiliki keterkaitan logis yang kuat guna mempermudah penarikan informasi melalui REST API nantinya.

3.1.1 Entity Relationship Diagram (ERD)

Untuk memvisualisasikan bagaimana aliran data dan keterkaitan antar informasi di dalam sistem, tim pengembang telah menyusun *Entity Relationship Diagram* (ERD). Diagram ini menjadi peta jalan bagi pengembangan tabel-tabel di dalam database agar tidak terjadi tumpang tindih data.



Berdasarkan diagram ERD di atas, dapat dilihat bahwa arsitektur data SatSet dibagi menjadi beberapa domain utama yang saling terintegrasi. Struktur ini memungkinkan sistem untuk memisahkan

antara data sensitif pengguna, data operasional pengaduan, hingga data publikasi informasi, sehingga kinerja server tetap optimal meskipun menangani ribuan transaksi secara bersamaan.

3.1.2 Daftar Tabel Lengkap dan Atribut

Berdasarkan rancangan ERD, berikut adalah rincian fungsional dari setiap tabel beserta atribut utamanya:

- **USER:** Menyimpan data identitas warga yang mencakup `id_user` sebagai primary key, serta `nik`, `nama_lengkap`, `email`, dan `password`.
- **KATEGORI_LAYANAN:** Berfungsi sebagai pengklasifikasi jenis layanan, memisahkan antara kategori "BIROKRASI" dan "PENGADUAN".
- **ANTREAN_BIROKRASI:** Mencatat pengajuan surat-menyurat warga, memiliki atribut `nomor_antrian`, `jenis_surat`, `status_antrian`, dan `waktu_pengajuan`.
- **LAPORAN_USER:** Digunakan untuk menampung keluhan fasilitas publik dengan atribut `judul_laporan`, `deskripsi`, `lampiran_url` untuk bukti foto, serta `status_laporan`.
- **ADMIN:** Menyimpan data petugas yang memiliki otoritas untuk memproses laporan dan mempublikasikan informasi.
- **TINDAK_LANJUT_LAPORAN:** Tabel ini merekam respons admin terhadap laporan warga, termasuk `catatan_admin` dan `waktu_tindak`.
- **NOTIFIKASI:** Mengelola pesan pemberitahuan untuk warga terkait pembaruan status layanan mereka.
- **MADING_DIGITAL:** Wadah bagi admin untuk mempublikasikan pengumuman atau konten informasi publik kepada seluruh warga.

3.1.3 Penjelasan Relasi Antar Tabel (One-to-Many & Many-to-Many)

Sistem ini mengimplementasikan relasi yang ketat untuk menjamin kaitan data yang logis:

- **One-to-Many (USER ke ANTREAN/LAPORAN/NOTIFIKASI):** Satu pengguna dapat mengajukan banyak antrean birokrasi, membuat banyak laporan pengaduan, serta menerima berbagai notifikasi dari sistem.
- **One-to-Many (KATEGORI ke ANTREAN/LAPORAN):** Satu kategori layanan dapat membawahi banyak transaksi antrean maupun laporan keluhan warga.
- **One-to-Many (ADMIN ke MADING/TINDAK_LANJUT):** Seorang admin memiliki otoritas untuk mempublikasikan banyak konten di mading digital dan melakukan proses tindak lanjut pada berbagai laporan yang masuk.
- **One-to-One / One-to-Many (LAPORAN ke TINDAK_LANJUT):** Setiap satu laporan warga akan ditangani melalui satu atau lebih entri tindak lanjut untuk memastikan masalah tersebut terselesaikan secara tuntas.

3.2 Desain OOP

Perancangan berorientasi objek dalam sistem SatSet mengadopsi standar industri *Layered Architecture* menggunakan Spring Boot. Pendekatan ini memisahkan aplikasi ke dalam lapisan-lapisan logis yang berbeda guna meningkatkan keterbacaan kode dan memudahkan proses pengembangan fitur di masa depan. Dengan menerapkan prinsip-prinsip utama OOP, sistem ini mampu merepresentasikan proses birokrasi dunia nyata ke dalam struktur objek Java yang modular.

3.2.1 Class Diagram (Minimal 5 Class)

Struktur internal aplikasi digambarkan melalui *Class Diagram* yang mendefinisikan kontrak kerja antar komponen, mulai dari penerimaan permintaan pengguna hingga penyimpanan permanen ke dalam basis data.



Diagram kelas di atas menunjukkan interaksi antar komponen inti seperti pengontrol, layanan, dan repositori. Struktur ini menjamin bahwa setiap kelas memiliki tanggung jawab tunggal yang jelas,

sesuai dengan prinsip *Single Responsibility Principle*, sehingga meminimalisir risiko kesalahan saat dilakukan pembaruan kode.

3.2.2 Penjelasan Hubungan Antar Class (Inheritance & Association)

Dalam implementasi teknisnya, sistem ini memanfaatkan pilar *Inheritance* (pewarisan) melalui penggunaan kelas abstrak *BaseEntity*. Kelas ini bertindak sebagai induk yang mewariskan atribut audit seperti *id* dan *createdAt* kepada kelas entitas seperti *TiketLayanan*, sehingga menghindari penulisan kode yang berulang. Selain itu, hubungan *Association* dijalankan melalui mekanisme *Dependency Injection*, di mana *TiketController* berasosiasi dengan *TiketService* untuk memproses logika bisnis, dan *TiketServiceImpl* berasosiasi dengan *TiketRepository* untuk akses data. Hubungan ini bersifat longgar (*loose coupling*) berkat penggunaan *interface*, yang memungkinkan fleksibilitas tinggi dalam pengembangan sistem.

3.2.3 Identifikasi Komponen Spring (@Entity, @Service, @Controller, @Repository)

Setiap kelas dalam diagram di atas dipetakan ke dalam komponen Spring Boot menggunakan anotasi stereotip yang spesifik. Anotasi *@RestController* digunakan pada kelas pengontrol untuk menangani masuknya permintaan HTTP dan mengembalikan respons JSON. Komponen *@Service* digunakan sebagai wadah logika bisnis utama, tempat di mana aturan-aturan administrasi diproses secara mendalam. Untuk interaksi data, digunakan anotasi *@Repository* yang memberikan abstraksi tingkat tinggi atas kueri database. Terakhir, anotasi *@Entity* digunakan untuk menandai kelas model yang merepresentasikan tabel di database, di mana prinsip *Encapsulation* diterapkan melalui penggunaan *getter* dan *setter* guna menjaga keamanan dan validitas status objek.

3.3 Rancangan REST API

Bagian terakhir dari perancangan sistem adalah pembuatan spesifikasi antarmuka pemrograman yang menjadi jembatan antara server dengan pengguna. API ini dirancang agar dapat diakses oleh berbagai platform dengan performa yang cepat dan standar keamanan yang mumpuni.

3.3.1 Daftar Endpoint REST API (Minimal 8 Endpoint)

Sistem SatSet menyediakan minimal delapan *endpoint* fungsional untuk mencakup seluruh siklus pelayanan publik. Daftar ini mencakup *GET /api/pengajuan* untuk memantau semua data, *GET /api/pengajuan/{id}* untuk detail spesifik, serta *POST /api/tickets/bureaucracy* dan *POST /api/tickets/complaints* untuk pengiriman data baru oleh warga. Tersedia juga *endpoint* untuk melihat antrean hari ini, mengambil data mading digital, hingga pembaruan status laporan oleh admin melalui jalur */api/tickets/{id}/resolve*.

3.3.2 Deskripsi Metode, URL, dan Parameter Endpoint

Setiap *endpoint* beroperasi menggunakan metode HTTP yang merepresentasikan aksi tertentu; metode *GET* digunakan untuk penarikan informasi, sedangkan *POST* dan *PUT* digunakan untuk pengiriman dan pembaruan data. Parameter identitas seperti *{id}* atau nomor

tiket dikirimkan melalui jalur URL untuk memastikan identifikasi data dilakukan secara unik. Seluruh permintaan yang masuk melalui URL tersebut akan divalidasi oleh *backend* guna memastikan parameter yang dikirimkan sesuai dengan format JSON yang diharapkan, sehingga integritas data yang masuk ke dalam sistem tetap terjaga sepenuhnya dari segala bentuk kesalahan input.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan seluruh tahapan pengembangan yang telah dilaksanakan, mulai dari tahap perencanaan hingga perancangan sistem, dapat disimpulkan bahwa proyek "SatSet Superapp" telah berhasil dirancang sebagai solusi digital untuk mengatasi kompleksitas birokrasi pelayanan masyarakat. Implementasi teknologi *backend* berbasis Java 17 dan Spring Boot terbukti mampu menyediakan fondasi yang kokoh dalam membangun REST API yang cepat, efisien, dan terstruktur. Melalui penerapan prinsip-prinsip *Object-Oriented Programming* (OOP) seperti *Encapsulation*, *Inheritance*, *Polymorphism*, dan *Abstraction*, kode program yang dihasilkan menjadi lebih modular dan mudah untuk dikembangkan di masa mendatang.

Selain itu, perancangan basis data relasional menggunakan PostgreSQL telah berhasil mengintegrasikan berbagai modul layanan, mulai dari pengaduan warga hingga antrean birokrasi, ke dalam satu sistem terpusat. Penggunaan infrastruktur modern seperti Proxmox VE dan Cloudflare Tunnel memberikan jaminan bahwa sistem ini tidak hanya dapat berfungsi dengan baik di lingkungan lokal, tetapi juga dapat diakses secara publik dengan tingkat keamanan yang terjaga. Secara keseluruhan, sistem SatSet siap menjadi platform yang mempermudah warga dalam berinteraksi dengan instansi pelayanan publik secara transparan dan *real-time*.

4.2 Saran

Meskipun sistem SatSet ini telah memenuhi kebutuhan dasar administrasi terpadu, tim pengembang menyadari bahwa masih terdapat ruang untuk pengembangan lebih lanjut guna mencapai kesempurnaan fungsionalitas. Oleh karena itu, terdapat beberapa saran yang dapat diajukan untuk pengembangan di masa depan:

Pertama, dari sisi teknis, sistem ini dapat ditingkatkan dengan menambahkan lapisan keamanan yang lebih kompleks seperti implementasi *Spring Security* dengan protokol JWT (*JSON Web Token*) untuk memastikan setiap permintaan API benar-benar berasal dari pengguna yang terverifikasi. Kedua, integrasi dengan layanan pihak ketiga seperti *Payment Gateway* untuk pengurusan pajak daerah atau *SMS/WhatsApp Gateway* untuk notifikasi langsung ke ponsel warga akan meningkatkan pengalaman pengguna secara signifikan. Terakhir, pengembangan antarmuka pengguna (*Frontend*) baik berbasis *web* maupun *mobile* perlu segera dilakukan agar masyarakat umum dapat mulai merasakan manfaat dari REST API yang telah dibangun oleh Kelompok 4 ini. Dengan adanya pengembangan berkelanjutan, diharapkan SatSet dapat benar-benar menjadi aplikasi andalan dalam mewujudkan tata kelola birokrasi yang modern di Indonesia.

